

Timéo BERNARD GOARANT

Lycée Vauban, Brest

Gatien JAUMOUILLE

Nolan RIBETTE

BTS CIEL IR



Drone cible d'entraînement pour la marine nationale



X



Sebastien GUIVARCH

Denis CAILLE

Francesa BREHONNET

Timéo BERNARD GOARANT

Gatien JAUMOUILLE

Nolan RIBETTE

1. Introduction	3
1.1 Contexte du projet	3
1.2 Objectif du projet	3
1.3 Intérêt du projet	4
2. Présentation des outils du projet	4
2.1 GitHub	4
2.2 Google Drive	5
2.3 Synthèse	5
3. Environnement de développement	6
3.1 Justification des outils et langages choisis	6
3.2 Justification des librairies utilisées	6
3.3 Identification des contraintes du client	8
4. Architecture du projet	8
4.1 Diagramme d'architecture logicielle et matérielle	8
4.2 Schéma réseau commenté	10
5. Développement – état d'avancement	11
5.1 Présentation des composants développés	11
5.2 Extrait de code commentés et expliqués	12
5.3 Tableau ou cahier de tests unitaires	18

1.Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du BTS CIEL option A au lycée Vauban de Brest, en partenariat avec la Marine nationale et l'IRENAV (Institut de Recherche de l'École Navale).

Il répond à un besoin opérationnel réel de la Marine : entraîner les équipages à faire face à des menaces navales modernes.

Depuis plusieurs années, notamment en mer Rouge, des attaques sont menées contre des navires militaires ou civils à l'aide de petits navires rapides (skiffs) ou de drones de surface autonomes (aussi appelé USV : Unmanned Surface Vehicle). Ces menaces sont peu coûteuses, rapides, difficiles à détecter et représentent un risque important.

Dans ce contexte, la Marine nationale souhaite disposer de drones cibles réalistes, capables de simuler ce type d'attaque lors d'exercices. Le projet qui nous est confié consiste donc à concevoir un prototype de drone naval de surface, à faible coût, destiné à être utilisé lors d'entraînements.

1.2 Objectif du projet

L'objectif principal du projet est de concevoir et réaliser un prototype fonctionnel de drone naval de type USV servant de cible d'entraînement pour la Marine nationale.

Plus précisément, le drone doit être capable de :

- Être piloté à distance par un opérateur (mode manuel),
- Suivre un itinéraire autonome à partir de points GPS (mode autonome),
- Se verrouiller sur une cible détectée par caméra et se diriger vers elle (mode verrouillage),
- Fonctionner avec des technologies peu coûteuses et des composants du commerce (approche DIY),
- Être suffisamment fiable pour des essais réels tout en restant économique, car il est conçu pour être potentiellement détruit lors des exercices.

Ce projet sert également de preuve de concept, afin d'évaluer la faisabilité de solutions low-cost pour des applications d'entraînement militaire.

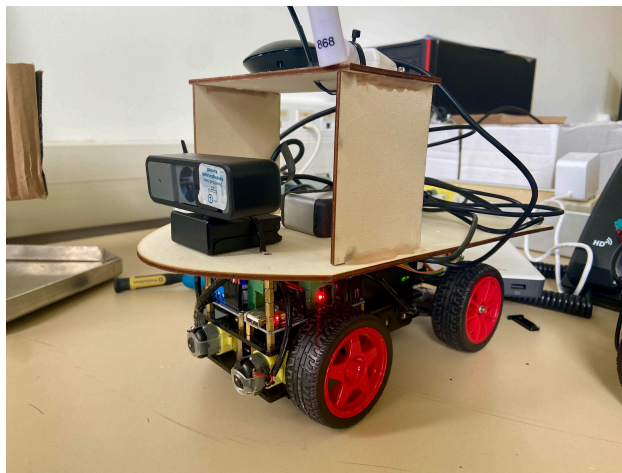


Photo du Drone Cible

1.3 Intérêt du projet

Ce projet présente un triple intérêt : opérationnel, technologique et pédagogique.

- Intérêt opérationnel
 - Il répond à un besoin concret de la Marine nationale en matière d'entraînement face à des menaces modernes.
 - Il permet de simuler des scénarios réalistes sans mobiliser des équipements coûteux.
 - Il contribue à améliorer la préparation et la réactivité des équipages.
- Intérêt technologique
 - Le projet mobilise des technologies actuelles : ROS 2, GPS, vision par caméra, communication LoRa, Docker.
 - Il explore l'utilisation de solutions low-cost dans un contexte militaire, ce qui est stratégique.
- Intérêt pédagogique
 - Il nous permet, étudiants, de travailler sur un projet réel et professionnalisant.
 - Il développe des compétences clés du BTS CIEL :
conception système, développement logiciel, communication, tests, documentation et travail en équipe.
 - Il met les étudiants en situation proche de l'ingénierie industrielle et de la recherche appliquée.

2. Présentation des outils du projet

2.1 GitHub

GitHub est utilisé comme outil central de gestion du projet. Il permet de centraliser l'ensemble du code source, d'assurer le travail collaboratif entre les membres de l'équipe et de conserver l'historique des modifications ainsi qu'un suivi du projet. Grâce au système de versions, chaque évolution du projet est tracée et sécurisée, ce qui facilite la correction des erreurs et la validation des développements. GitHub est également utilisé pour héberger la documentation technique, garantissant ainsi la continuité, la fiabilité et la maintenabilité du projet.

Dans le GitHub, nous avons créé une arborescence simple et intuitive.

- SysML : contient les différents SysML du projet.
- Simulation : contient les scripts de tests et de simulation du projet.
- Docs technique : l'entièreté des documentation technique des membres du groupe.
- Code : vous y trouverez le code source du projet.

Les fonctionnalités de gestion de projet de GitHub ont été utilisées pour organiser et suivre l'avancement du travail. Les issues permettent de décrire les tâches à réaliser, les anomalies à corriger ou les améliorations à apporter, tout en les attribuant à un membre de l'équipe. Le tableau Kanban offre une vision claire de l'état du projet en classant les tâches selon leur

progression (à faire, en cours, terminé). Le travail est découpé en sprints, correspondant à des périodes de développement définies, durant lesquelles des objectifs précis sont fixés. Cette organisation permet un suivi rigoureux, une meilleure répartition des tâches et une adaptation rapide en cas de difficulté.

2.2 Google Drive

Google Drive est utilisé comme espace de stockage et de partage des documents du projet. Il permet de centraliser les fichiers non liés au code, tels que les documents de conception, les comptes rendus de réunion, et les schémas. Cet outil facilite le travail collaboratif en assurant un accès commun aux informations à jour, tout en évitant les doublons et les pertes de données. Google Drive contribue ainsi à une organisation efficace et à la continuité du projet.

Voici son arborescence :

- Compte rendu : contient les comptes rendu des différents sprints.
- Schémas : les différents schémas en lien avec notre projet.
- Soutenance : contient des documents utiles pour la rédaction de la soutenance, le rapport commun ainsi que 3 dossiers :
 - Timéo
 - Gatien
 - Nolan

Ils contiennent les rédactions de soutenance de chaque membre du groupe.

Google Drive nous sert bien uniquement à déposer des documentations ainsi que des schémas. Vous n'y trouverez en aucun cas des codes dedans.

2.3 Synthèse

Cette organisation a été choisie afin de séparer clairement le développement logiciel de la gestion documentaire. GitHub est dédié au code source, au suivi des tâches et à la collaboration technique, tandis que Google Drive est utilisé pour le partage et la centralisation des documents du projet. Cette complémentarité facilite le travail en équipe en offrant à chacun un accès rapide aux informations nécessaires, réduit les risques d'erreurs ou de doublons et améliore la coordination entre les membres du groupe. Elle permet ainsi un suivi structuré, efficace et collaboratif du projet.

3. Environnement de développement

3.1 Justification des outils et langages choisis

Dans le cadre du projet Drone Cible, le développement logiciel a été réalisé principalement en Python. Ce langage a été choisi pour plusieurs raisons. Tout d'abord, Python est un langage simple à utiliser et très lisible, ce qui facilite la maintenance du code et la collaboration entre les différents membres de l'équipe. De plus, Python possède de nombreuses bibliothèques permettant d'interagir facilement avec des capteurs et des interfaces matérielles.

Le projet utilise également ROS2 (Robot Operating System 2). ROS2 est une plateforme logicielle largement utilisée dans le domaine de la robotique. Elle permet d'organiser le programme sous forme de nœuds logiciels qui communiquent entre eux grâce à des messages. Cette architecture modulaire facilite le développement et la maintenance du système, car chaque composant du projet peut être développé et testé indépendamment.

Dans la partie du mode autonome du projet Drone Cible, plusieurs nœuds ROS2 sont utilisés pour :

- la lecture du capteur IMU
- la lecture des données GPS
- la gestion de la navigation

Afin de garantir un environnement de développement stable, l'ensemble du système est exécuté dans un conteneur Docker. Docker permet d'isoler l'environnement logiciel et d'installer toutes les dépendances nécessaires au projet dans un conteneur unique. Cette solution améliore également la reproductibilité du projet et simplifie les mises à jour du système.

3.2 Justification des librairies utilisées

Pour faciliter le développement du système du drone cible, plusieurs bibliothèques Python et dépendances système ont été utilisées. Ces librairies permettent d'assurer la communication avec les capteurs, le traitement des données et l'intégration avec l'environnement ROS2.

Dans l'environnement Docker utilisé pour le mode autonome, plusieurs dépendances ont été installées afin de garantir le bon fonctionnement des différents modules du projet.

Parmi ces dépendances, on retrouve notamment :

- python3-pip : permet l'installation et la gestion des bibliothèques Python
- python3-colcon-common-extensions : outil utilisé pour compiler et gérer le projet ROS2.
- ros-humble-rmw-cyclonedds-cpp : middleware utilisé par ROS2 pour la communication entre les différents nœuds du système.
- gpsd et gpsd-clients : outils permettant de gérer et d'accéder aux données provenant du module GPS.
- python3-gps : bibliothèque permettant d'interagir avec le service gpsd.
- iproute2 et iputils-ping : outils réseau permettant de tester la connectivité et de diagnostiquer les communications réseau.

- nano : éditeur de texte utilisé pour modifier les fichiers de configuration directement dans le conteneur.

Certaines bibliothèques Python spécifiques ont également été installées via pip afin d'interagir avec les capteurs utilisés dans le projet.

Le capteur IMU (Inertial Measurement Unit) permet de mesurer différents paramètres physiques du drone tels que l'accélération, la rotation et le champ magnétique. Ces données sont utilisées pour estimer l'orientation et les mouvements du drone.

Les librairies suivantes sont utilisées pour la gestion de ce capteur :

- rclpy : bibliothèque Python permettant d'interagir avec ROS2. Elle permet de créer des nœuds ROS2 et de gérer la communication entre les différents composants du système.
- sensor_msgs.msg (Imu, MagneticField) : ensemble de messages standard fournis par ROS2 permettant de transmettre les données provenant des capteurs inertiels.
- smbus2 : bibliothèque utilisée pour la communication avec le capteur IMU via le bus I2C. Elle permet de lire et d'écrire des données dans les registres du capteur.
- math : utilisée pour effectuer les calculs mathématiques nécessaires au traitement des données du capteur.
- time : utilisée pour gérer les temporisations et contrôler la fréquence de lecture du capteur.

Ces bibliothèques permettent de récupérer les données du capteur IMU et de les publier dans le réseau ROS2 afin qu'elles puissent être utilisées par les autres composants du système.

Le drone utilise également un module GPS afin de déterminer sa position géographique. Ces informations sont utilisées pour suivre un itinéraire composé de plusieurs points appelés waypoints.

Les waypoints sont définis dans un fichier GeoJSON contenant les coordonnées en latitude et en longitude.

Les bibliothèques suivantes sont utilisées pour gérer les données GPS :

- rclpy : permet d'intégrer le module GPS dans l'architecture ROS2 sous forme de nœud.
- geometry_msgs.msg.Point : message ROS permettant de représenter une position dans l'espace.
- json : bibliothèque utilisée pour lire le fichier GeoJSON contenant les coordonnées des points de passage.
- serial : permet la communication avec le module GPS via une interface série.
- pynmea2 : bibliothèque permettant de décoder les trames NMEA envoyées par le module GPS afin d'extraire les informations de position telles que la latitude et la longitude.

Grâce à ces différentes bibliothèques, il est possible de récupérer les coordonnées GPS du drone, de traiter les données et de les transmettre aux autres nœuds du système via ROS2.

3.3 Identification des contraintes du client

Le mode autonome doit respecter plusieurs contraintes définies dans le cahier des charges.

Le système de navigation doit être capable de suivre un itinéraire avec une précision d'environ ± 5 mètres. Cette contrainte impose une utilisation fiable du module GPS ainsi qu'un traitement correct des données de position.

Le système doit permettre de définir facilement un itinéraire pour le drone. Pour cela, les points de navigation sont stockés dans un fichier GeoJSON avec les coordonnées en latitude et en longitude.

Cette solution permet de modifier facilement le trajet du drone sans avoir à modifier directement le programme.

Plusieurs dispositifs de sécurité doivent être intégrés au système afin d'éviter un comportement dangereux du drone.

Tout d'abord, si le signal GPS ou la communication est perdue pendant une durée supérieure à 3 secondes, le mode autonome est automatiquement désactivé.

De plus, un bouton d'arrêt d'urgence est prévu afin de pouvoir arrêter immédiatement le fonctionnement du drone en cas de problème.

4. Architecture du projet

4.1 Diagramme d'architecture logicielle et matérielle

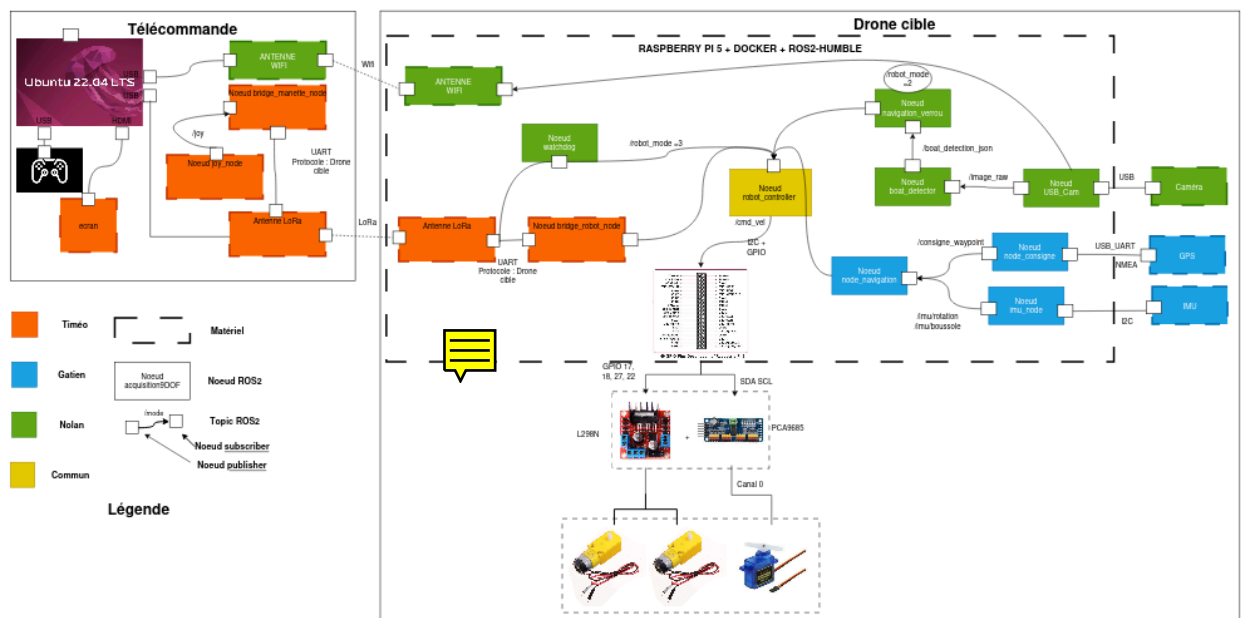


Diagramme d'architecture logicielle et matérielle du projet drone cible

Voici ci-dessous le diagramme d'architecture logicielle et matérielle du mode autonome :

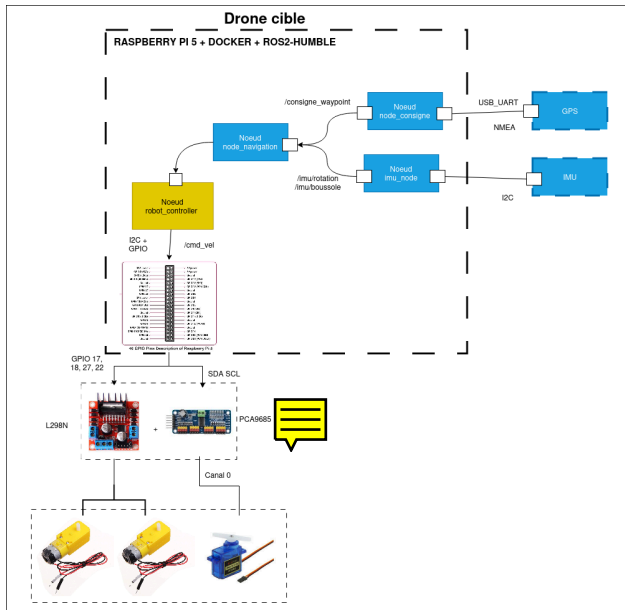


Diagramme d'architecture logicielle et matérielle du mode autonome

Ce diagramme présente l'architecture du mode autonome. Le système est basé sur un Raspberry Pi 5 exécutant ROS2 dans le conteneur Docker du mode autonome (ros2_auto). Les différents programmes sont organisés sous forme de nœuds ROS2 qui communiquent entre eux.

Le nœud node_consigne récupère les trames NMEA du GPS via une communication USB UART, tandis que le nœud imu_node lit les données du capteur IMU via le bus I2C. Ces informations sont ensuite traitées par le nœud node_navigation, qui calcule les commandes de déplacement.

Les commandes de déplacement sont envoyées au topic /cmd_vel. Le nœud robot_controller, lorsqu'il est actif, s'abonne à ce topic et utilise ces commandes pour contrôler les actionneurs via les interfaces GPIO et I2C du Raspberry Pi. Les moteurs sont pilotés par un driver L298N, tandis qu'un module PCA9685 permet de contrôler le servomoteur.

4.2 Schéma réseau commenté

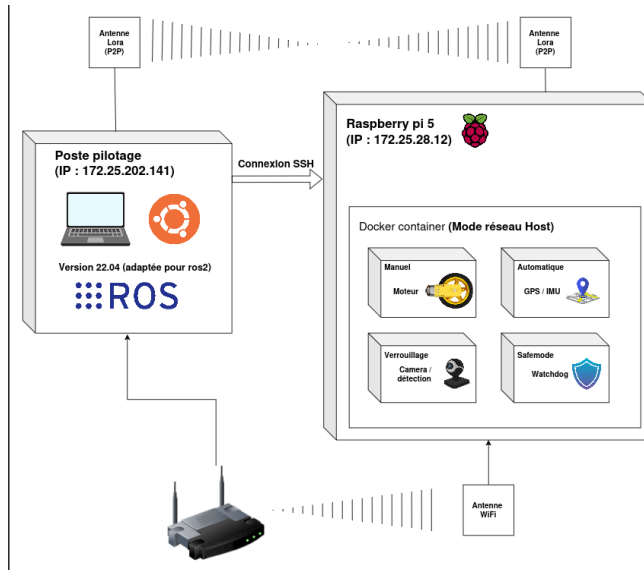


Schéma du réseau du projet drone cible

Le capteur GPS communique avec le Raspberry Pi via une interface UART (USB / série) et transmet des trames NMEA contenant les informations de position, de la latitude et de la longitude.

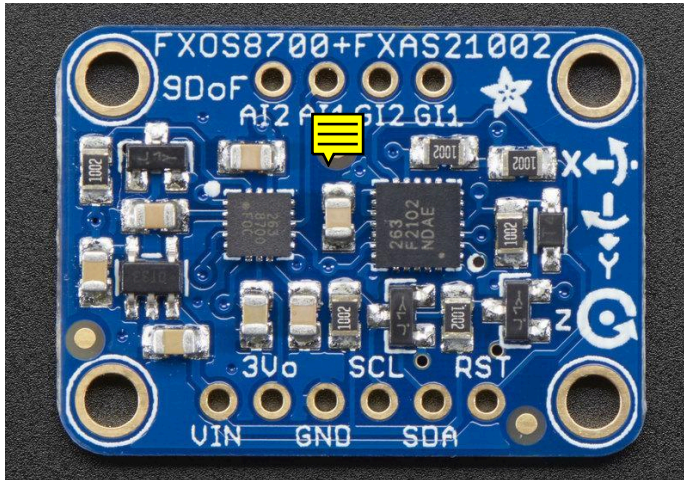
Le capteur IMU est connecté au bus I2C et envoie des données sur l'accélération, la rotation (gyroscope) et l'orientation (boussole).

Ces données sont récupérées par les nœuds ROS2. Elles sont ensuite transmises via une liaison radio LoRa (P2P) grâce aux antennes LoRa présentes sur le drone et sur le poste de pilotage.

Les informations reçues sur le poste de pilotage sont ensuite utilisées par une interface Node-RED, permettant d'afficher en temps réel les données de position GPS sur une carte, la rotation sur un inclinomètre et l'orientation avec une boussole pour l'opérateur.

5. Développement – état d'avancement

5.1 Présentation des composants développés



Capteur Adafruit IMU 9DoF FXOS8700 + FXAS21002

Le capteur NXP Precision 9DoF Breakout est un module IMU (Inertial Measurement Unit) utilisé pour mesurer les mouvements et l'orientation d'un système. Il combine plusieurs capteurs permettant d'obtenir des informations précises sur l'accélération, la rotation et le champ magnétique.

Ce module intègre deux composants principaux :

- FXOS8700 : un capteur combinant un accéléromètre 3 axes et un magnétomètre 3 axes.
- FXAS21002 : un gyroscope 3 axes.

Grâce aux capteurs, le module fournit 9 degrés de liberté (9DoF), ce qui permet de mesurer les mouvements dans l'espace selon les axes X, Y et Z. Il peut ainsi déterminer l'orientation et les déplacements d'un objet.



Capteur GPS BU-353

Le GPS BU-353 se connecte en USB à la Raspberry. Il transmet les données de position sous forme de trames NMEA, qui contiennent différentes informations comme la latitude, la longitude, l'altitude, la vitesse et l'heure GPS.

Ce module est reconnu pour sa bonne sensibilité de réception et sa capacité à capter plusieurs satellites qui améliore la précision de localisation. Grâce à ces caractéristiques, il est particulièrement adapté pour des applications de géolocalisation et de navigation.

Dans le projet du drone cible, ce capteur est utilisé pour récupérer les coordonnées GPS du robot et permettre au système de navigation de suivre un itinéraire défini par des points de passage (waypoints).

5.2 Extrait de code commentés et expliqués

Dans le projet du Drone Cible, le nœud ROS2 (imu_publisher) a été développé afin de récupérer, traiter et transmettre les données du capteur IMU. Ce capteur permet de mesurer l'accélération, la rotation et le champ magnétique, pour déterminer l'orientation du drone.

Création du nœud ROS2 et des publishers :

```
class IMUPublisher(Node):
    def __init__(self, calib_samples=200):
        super().__init__('imu_publisher')

        self.imu_pub = self.create_publisher(Imu, 'imu/data_raw', 10)
        self.mag_pub = self.create_publisher(MagneticField, 'imu/mag', 10)
        self.boussole_pub = self.create_publisher(String, 'imu/boussole_bot', 10)
```

Ce code permet de créer un nœud ROS2 nommé `imu_publisher`.

Trois publishers sont définis afin de diffuser les données sur différents topics :

- `/imu/data_raw` : données d'accélération et de rotation
- `/imu/mag` : données du magnétomètre
- `/imu/boussole_bot` : direction du drone (boussole)

Cette organisation permet aux autres nœuds ROS2 de s'abonner aux données nécessaires en fonction de leurs besoins.

Communication en I2C avec le capteur IMU :

```
self.bus = smbus2.SMBus(1)
```

```
self.ADDR_ACCEL_MAG = 0x1F
```

```
self.ADDR_GYRO = 0x21
```

Cette partie initialise la communication avec le capteur via le bus I2C du Raspberry Pi.

- `SMBus(1)` correspond au bus I2C principal
- `0x1F` est l'adresse du capteur accéléromètre et magnétomètre
- `0x21` est l'adresse du gyroscope

Ces adresses permettent au programme de communiquer directement avec les composants matériels du capteur.

Calibration du capteur :

```
def calibrate(self, samples=200):
```

```
    for _ in range(samples):
```

```
        accel, _ = self.read_accel_mag()
```

```
        gyro = self.read_gyro()
```

La calibration permet de corriger les erreurs de mesure du capteur.

Le principe consiste à :

1. effectuer plusieurs mesures lorsque le capteur est immobile
2. calculer la moyenne des valeurs obtenues
3. utiliser cette moyenne comme offset de correction

Ces offsets sont ensuite soustraits aux mesures en temps réel, ce qui permet d'améliorer la précision des données.

Calcul de l'orientation (boussole) :

```
heading_rad = math.atan2(mag[1], mag[0])
```

```
heading_deg = math.degrees(heading_rad)
```

```
if heading_deg < 0:
```

```
    heading_deg += 360
```

Cette partie permet de calculer le cap du drone à partir des données du magnétomètre.

- `atan2()` permet de calculer un angle à partir des axes X et Y
- conversion en degrés
- ajustement pour obtenir une valeur entre 0° et 360°

Une fonction complémentaire permet ensuite de convertir cet angle en direction cardinale (Nord, Est, Sud, Ouest).

Publication des données dans ROS2 :

```
imu_msg = Imu()
```

```
imu_msg.linear_acceleration.x = accel[0] * 9.80665
```

```
imu_msg.angular_velocity.x = math.radians(gyro[0])
```

```
self.imu_pub.publish(imu_msg)
```

Ce code permet de :

1. créer un message ROS2 de type Imu
2. insérer les données du capteur après conversion :
 - accélération en m/s²
 - vitesse angulaire en radians/s
3. publier ces données sur le topic `/imu/data_raw`

Les autres nœuds du système peuvent ainsi exploiter ces informations pour la navigation ou la stabilisation.

Publication de la direction (boussole) :

```
boussole_msg = String()
```

```
boussole_msg.data = f"{heading_deg:.1f}° - {direction}"
```

```
self.boussole_pub.publish(boussole_msg)
```

Cette partie permet d'envoyer une information lisible par l'utilisateur, indiquant le cap du drone sous forme d'angle et de direction.

Exemple :

120.5° - Sud-Est

Ces données peuvent être utilisées pour un affichage sur une interface Node-RED.

Exécution périodique du programme :

```
self.timer = self.create_timer(0.5, self.timer_callback)
```

Cette instruction permet d'exécuter le programme toutes les 0,5 secondes (2 Hz).

À chaque cycle :

- les données du capteur sont lues
- elles sont corrigées (offset)
- elles sont publiées dans ROS2

Cela permet d'obtenir un flux de données en temps réel.

Ce programme permet de mettre en place un nœud ROS2 complet capable de :

- communiquer avec un capteur IMU via le bus I2C
- corriger les erreurs de mesure grâce à une calibration
- calculer l'orientation du drone
- publier les données sur plusieurs topics ROS2

Ces informations sont essentielles pour assurer le pilotage, la stabilisation et la navigation autonome du drone.

Pour le GPS du projet Drone Cible, le nœud ROS2 (node_consigne) a été développé afin de gérer la navigation du drone à partir de points de passage (waypoints). Ce nœud récupère la position GPS du drone, compare cette position avec les waypoints définis et envoie les consignes de navigation au système.

Initialisation du nœud et des paramètres :

```
class NoeudConsigne(Node):
```

```
    def __init__(self):
```

```
        super().__init__('node_consigne')
```

```
        self.declare_parameter("geojson_file", "/root/ros_ws/Waypoints/waypoints.json")
```

```
        self.declare_parameter("gps_port", "/dev/ttyUSB0")
```

```
        self.declare_parameter("baudrate", 4800)
```

```
self.declare_parameter("tolerance", 5.0)
```

Ce code permet de créer un nœud ROS2 nommé `node_consigne`.

Plusieurs paramètres sont définis :

- `geojson_file` : chemin vers le fichier contenant les waypoints
- `gps_port` : port de communication du GPS
- `baudrate` : vitesse de communication série
- `tolerance` : distance (en mètres) pour considérer un waypoint atteint

Ces paramètres permettent de rendre le système configurable sans modifier le code.

Chargement des waypoints (GeoJSON) :

```
def load_waypoints(self, file):
```

```
    with open(file, 'r') as f:
```

```
        data = json.load(f)
```

Cette fonction permet de charger les waypoints depuis un fichier GeoJSON.

```
waypoints.append({
```

```
    "order": order,
```

```
    "lat": lat,
```

```
    "lon": lon
```

```
})
```

Chaque waypoint contient :

- une latitude
- une longitude
- un ordre de passage

```
waypoints.sort(key=lambda x: x["order"])
```

Les points sont ensuite triés afin de garantir que le drone suit le bon itinéraire.

Cette méthode permet de modifier facilement le trajet du drone sans modifier le programme.

Lecture des données GPS :

```
self.ser = serial.Serial(gps_port, baudrate, timeout=1)
```

Cette ligne initialise la communication avec le module GPS via une liaison série (UART/USB).

```
line = self.ser.readline().decode('ascii', errors='replace')
```

Le programme lit les données envoyées par le GPS sous forme de trames NMEA.

```
msg = pynmea2.parse(line)
```

La bibliothèque pynmea2 permet de décoder ces trames pour extraire :

- la latitude
- la longitude

Cette étape permet d'obtenir la position actuelle du drone en temps réel.

Calcul de la distance GPS :

```
def distance_gps(self, lat1, lon1, lat2, lon2):
```

```
    R = 6371000
```

Cette fonction calcule la distance entre deux points GPS.

```
    return R*c
```

Elle utilise la formule de Haversine, qui permet de calculer une distance sur la surface de la Terre en tenant compte de sa courbure.

Le résultat est une distance en mètres entre le drone et le waypoint.

Publication de la consigne vers la navigation :

```
msg = Point()
```

```
msg.x = wp["lat"]
```

```
msg.y = wp["lon"]
```

```
msg.z = 0.0
```

```
self.publisher_.publish(msg)
```

Cette partie permet de créer un message ROS2 contenant la position du waypoint à atteindre.

Le message est publié sur le topic :

```
consigne_waypoint
```

Le nœud de navigation récupère cette consigne pour guider le drone vers la position.

Gestion de l'atteinte du waypoint :

```
if distance <= self.tolerance:
```

```
    self.get_logger().info("Waypoint atteint !")
```

```
self.index_courant += 1
```

Lorsque la distance entre le drone et le waypoint est inférieure à la tolérance définie (5 mètres), le waypoint est considéré comme atteint.

Le programme passe alors automatiquement au waypoint suivant.

Cela permet une navigation autonome étape par étape.

Boucle principale du programme :

```
self.timer = self.create_timer(1.0, self.main_loop)
```

Cette instruction permet d'exécuter la fonction principale toutes les 1 seconde.

À chaque cycle :

- récupération de la position GPS
- calcul de la distance au waypoint
- envoi de la consigne
- vérification si le waypoint est atteint

Cela permet un suivi en temps réel de la progression du drone.

Ce programme permet de créer un nœud ROS2 chargé de :

- lire la position du drone à partir du GPS
- charger un itinéraire depuis un fichier GeoJSON
- calculer la distance entre le drone et les waypoints
- envoyer les consignes de navigation au système
- gérer automatiquement le passage d'un waypoint à un autre

Ce nœud constitue un élément essentiel du système de navigation autonome du drone.

5.3 Tableau ou cahier de tests unitaires